

✖ Team Performance

View Leaderboard

 Academic Track	Rank	Best Score	Best Score Submission Time
	56	0.832284	2026-05-24 15:16:56

TAAC 最后两天

0.828 → 0.83228

DIN + MLP 架构分享

Rank 56 - Acloudsky

→ <https://github.com/Shichien/TAAC2026-DIN> ★

@诗千 Shichien

得益于 DIN MLP 这条更轻的路线，抽到好卡时可以达到 6 min / epoch，甚至能够有机会二分调参 `reinit_threshold`。以下代码均是全程 Vibe Coding 出来的，某些实现肯定并非最佳，仅供参考和学习，主包也是什么都不懂的。

一些小策略

1. 关于夜间训练 OOM，可以通过脚本自动检测 Job Failed，然后 POST Run 请求即可。
2. 在一个 idea 已经落地时趁早 AUC，如果成了，能尽早拿到更优的底座进行迭代。
3. 多进行文档记录，多做一些可观测的指标，如 PCOC、Brier 等指标，以选择更优的 Epoch，有时以前做的消融实验说不定哪天就能用上。我最后两天猛猛上分都得益于对数据的完整 EDA 和单模块消融，还有不能感觉两个策略能同时 Work 就一锅乱炖，尽量单个做消融。
4. 多尝试让 Agent 探索比赛平台，它几乎能够帮助你实现所有收集信息的需求，如我最后一张图中，是 Agent 自动帮我提交的最后一份 Eval。

最初 Baseline 架构优化尝试

AUC: 0.827931. 总体的 Timeline 如下:

1. 在 Baseline 基础上加入样本级绝对时间特征, 只保留 Weekday 和 Hour, 不做周期编码, 直接加入到 User Dense 侧, AUC 涨到 0.824471。入口先把 `sample_timestamps` 带进 `ModelInput`, 模型里根据时间戳构造 Hour 和 Weekday 的绝对时间上下文, 再把它整体加回 Non-Sequence Token。

`BestHyFormer/train/model.py`

```
def _build_absolute_time_context(
    self, sample_timestamps: torch.Tensor
) → torch.Tensor:
    shifted = (
        sample_timestamps.long()
        + int(self.timestamp_utc_offset_hours) * 3600
    )
    hour = torch.remainder(
        torch.div(shifted, 3600, rounding_mode="floor"), 24
    )
    day_index = torch.div(shifted, 86400, rounding_mode="floor")
    weekday = torch.remainder(day_index + 3, 7)
    is_weekend = (weekday ≥ 5).long()
    is_night = ((hour ≥ 23) | (hour ≤ 1)).long()

    context = (
        self.abs_hour_embedding(hour)
        + self.abs_weekday_embedding(weekday)
    )
    return self.abs_time_context_norm(context)

...
ns_tokens = ns_tokens + self._build_absolute_time_context(
    inputs.sample_timestamps
).unsqueeze(1)
```


2. 加入 Item Context, 但这没有一份完全干净的单点 AUC, 但反向消融能看出它提供过有效信息。在最终 `output` 上加一个 Item 侧 residual, 这个 residual 不是取全部 Item Dense, 而是对 Item NS Token 做均值后再投影一层。

`BestHyFormer/train/model.py`

```
def _build_item_context(self, item_ns: torch.Tensor) → torch.Tensor:
    return F.silu(self.item_context_proj(item_ns.mean(dim=1)))
...
output = output + self._build_item_context(item_ns)
```

3. 加入 User Dense 和 User Int 的同源 Pair 编码。只保留 62 到 66 的同源 Pair 对齐特征时, AUC 是 0.826989;

前者把原始 `user_dense` 按字段分别投影后再求均值融合, 后者把同一 `fid` 的 `user_int` embedding 和 `user_dense` 切片拼成 pair residual, 最后再加回 `output`。

`BestHyFormer/train/model.py`

```
tokens.append(F.silu(projection(dense_slice)))
...
return self.output_norm(torch.stack(tokens, dim=1).mean(dim=1))
```

4. 早期最强融合版把样本时间、User Dense 拆分、Item Context 和 Pair 对齐放在一起, AUC 是 0.827347。

5. 将 Dense 优化器改为 Muon, 保留稀疏 Embedding 继续走 Adagrad, 只把非 Embedding 的 Dense 参数切到 Muon。AUC 到 0.827931。但代价是训练速度变慢接近一倍。

BestHyFormer/train/trainer.py

```
def build_dense_optimizer(
    dense_params, *, dense_optimizer_type: str, lr: float
) → torch.optim.Optimizer:
    if dense_optimizer_type == "muon":
        return Muon(
            dense_params, lr=lr, weight_decay=0.0, adamw_betas=(0.9, 0.98)
        )
    if dense_optimizer_type == "adamw":
        return torch.optim.AdamW(
            dense_params, lr=lr, betas=(0.9, 0.98), foreach=False
        )
self.sparse_optimizer = torch.optim.Adagrad(sparse_params, lr=sparse_lr)
self.dense_optimizer = build_dense_optimizer(
    dense_params, dense_optimizer_type=dense_optimizer_type, lr=lr
)
```

到这里为止, HyFormer 主干再怎么加模块都很涨不上去了, 另一方面是由于主包浮躁无法等待长久的训练, 于是接下来进入 DIN 的搭建阶段。

搭起 DIN MLP 的底座

DIN 版本保留了原来数据读取、稀疏特征、Dense 特征和多域行为序列这些输入，但把序列主干从 HyFormer 的 Query Token 加 Block 交互，改成了 Candidate Item 作为 Query 的 DIN Attention。也就是说，原来是先得到非序列 Token 和四路序列 Token 放进 HyFormer Block 里交互；现在是先得到 `item_repr`，再用它分别去四路行为序列里做 Target Attention，最后把用户表示、物品表示、样本时间、活跃度和四路兴趣向量拼起来过 MLP。

1. 原来会被 `emb_skip_threshold` 跳过的部分高基数序列字段，改成显式 Hash Embedding 接回来。训练入口里指定了四个序列字段，模型侧在 `SequenceFeatureEncoder` 中对这些 Slot 走 Hash Embedding 分支。

`BestDIN/train/train.py`

```
DEFAULT_SEQ_HASH_EMBEDDING = (  
    "seq_b:69:65536:4,seq_c:29:65536:4,seq_c:34:65536:4,seq_c:47:65536:4"  
)
```

2. 把序列时间从单个历史年龄桶，扩展成更贴近 93 分钟预测窗口的多槽时间特征。HyFormer 里每个 Domain 只有 `time_bucket`，模型在序

列 Token 上加一个 `time_embedding`；DIN 里直接把 `age_bucket`、`hour`、`weekday`、`recent_window`、`horizon_age_bucket`、`horizon_window` 写入序列 Side Info，并在 Attention 分数上加入 Recency 和 Horizon Bias。

`BestDIN/train/dataset.py`

```
out[:, slots["age_bucket"], :] = age_bucket
out[:, slots["hour"], :] = hour
out[:, slots["weekday"], :] = weekday
out[:, slots["recent_window"], :] = recent_window
out[:, slots["horizon_age_bucket"], :] = horizon_age_bucket
out[:, slots["horizon_window"], :] = horizon_window
```

`BestDIN/train/model.py`

```
bias = bias + torch.where(
    valid_recent,
    recency_score * float(SEQ_RECENCY_ATTENTION_BIAS),
    bias,
)
bias = bias + torch.where(
    valid_horizon,
    horizon_score * float(SEQ_HORIZON_RECENCY_ATTENTION_BIAS),
    bias.new_zeros(bias.shape),
)
```

3. 把样本级时间和序列活跃度作为非序列上下文接入。原来的 `ModelInput` 只有 User、Item、Sequence 和 `seq_time_buckets`；现在 Batch 里额外返回 `sample_time_feats` 和 `activity_feats`，模型里分别通过

`SampleTimeEncoder` 和 `DenseFeatureEncoder` 接入最后的 MLP。

`BestDIN/train/model.py`

```
class ModelInput(NamedTuple):
    user_int_feats: torch.Tensor
    item_int_feats: torch.Tensor
    user_dense_feats: torch.Tensor
    item_dense_feats: torch.Tensor
    sample_time_feats: torch.Tensor
    activity_feats: torch.Tensor
    seq_data: Dict[str, torch.Tensor]
    seq_lens: Dict[str, torch.Tensor]
```

`BestDIN/train/dataset.py`

```
"sample_time_feats": torch.from_numpy(sample_time_feats.copy()),
"activity_feats": torch.from_numpy(activity_features.copy()),
```

4. User Dense 在 DIN 里也不是简单把所有 Dense 拼起来过一层 Linear，而是把 UE 类字段、同源 Int Dense Pair、较小 UE 字段分开编码后再融合。这里保留了 HyFormer 阶段已经验证过的同源 Pair 思路，只是接入到了 DIN 的 User 表示里。

`BestDIN/train/model.py`

```
return F.silu(
    self.fuse(
        torch.cat(
            [
                self.main_ue_encoder(dense_feats),
                self.pair_encoder(int_feats, dense_feats),
                self.small_ue_encoder(dense_feats),
            ],
            dim=-1,
```



```
)  
    )  
)
```

最终输出层也从 HyFormer 的单个 **output** 过分类器，变成 DIN 版本的多路表示拼接后过 MLP。这里拼进去的包括 User 表示、Item 表示、样本时间、活跃度、四路序列兴趣，以及全局兴趣和 Item 的乘积差分交互。

BestDIN/train/model.py

```
features = torch.cat(  
    [  
        user_repr,  
        item_repr,  
        self.sample_time_encoder(inputs.sample_time_feats),  
        self.activity_encoder(inputs.activity_feats),  
        *interest_parts,  
        global_interest * item_repr,  
        torch.abs(global_interest - item_repr),  
    ],  
    dim=-1,  
)  
return self.classifier(self.readout(features))
```

这条线最重要的结论是：真正带来线上提升的不是把模型做得更复杂，而是让模型围绕更稳定的泛化信号学习。最后的涨分更多来自时间分布对齐、序列高基数信号和更干净的输入口径，而不是加模块、加容量、加更多显式交互。

最终 BestDIN 这条线 AUC 是 0.832284.

一句话总结

DIN 最终涨分不是靠更大的模型，也不是靠更激进的时间权重，而是靠三个动作：接回真正有价值的序列高基数信息，把序列时间温和对齐到未来 93 分钟，再把输入口径收敛到更稳定的泛化信号上。最后的 0.832284，本质上是让模型学到更贴近 Public Test 的东西，而不是多堆东西。